

Name: Natalie Poche
Date: August 28, 2026
UFID: 1533-9442

Report - Equipment Demo

1 Warm-Up

Explain the terms volts per div and time per div as applied to an oscilloscope. Where specifically are they indicated on the scope's display?

Answer: The volts per div are the vertical scale of the oscilloscope. The time per div are the horizontal scale of the oscilloscope. Changes to these values can impact the view of the scope, in otherwords, the zoom in/zoom out portions of the scope.

2 Warm-Up

What's the green terminal on the power supply? What can it be used for and how, physically, could it be used?

Answer: The green terminal on the power supply is the true ground/chassis ground terminal. It can be used when a floating or relative voltage reference is not good enough. It can be physically used by connecting the channel to the circuit's ground node.

3 Technical

Explain, in your own words, how the scale settings for voltage time affect an oscilloscope's capturing of a waveform. (A good response should identify, describe, and explain the inner workings of an oscilloscope.)

Answer: The scale settings for voltage has two parts. The signal range scale for voltage controls the amplification applied to the input before it reaches the ADC. This determines the ADC's available range and impacts the vertical resolution. The signal range position for voltage impacts the analog front end where it controls the offset DAC that is summed with the input signal. If the signal is positioned too far off-screen, the input can exceed the ADC's range and become clipped, meaning the captured data is actually incorrect rather than merely being displayed incorrectly. The scale settings for time determine the oscilloscope's sampling rate which can impact the resolution of the waveform and the total amount of time included in the capture. A faster time scale allows the oscilloscope to capture rapid changes with greater resolution, while a slower time scale allows a larger time window to be observed.

4 Scenario

You are designing a class-D audio amplifier that needs a bipolar, +/-12V DC power supply for the MOSFETs, as well as a 5V DC supply to power to logic circuitry. Make a labelled diagram showing how this could be done with power supplies available in our lab (note: all physical connections should be shown!).

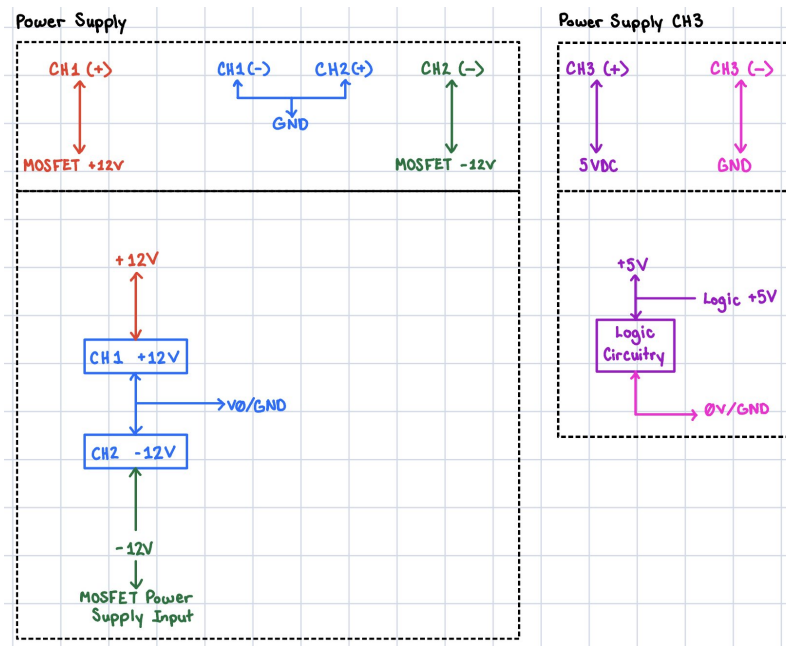


Figure 1: Audio Amplifier Diagram (image0.jpg)

5 Scenario

Rob received a synth PCB he designed by mail. He assembled the board and baked it in the reflow oven. He plugged it in and... pop! The power supply was reversed. Oops!!

5.1 Scenario

Briefly describe how a programmable power supply and other equipment could detect which component(s) were damaged with minimal risk (hint: faulty components get hot). (Must be practical.)

Answer: A practical approach for Rob would be to use the programmable power supply with a very low current limit then gradually increase the voltage while monitoring the supply current on the board. Look for components that become unusually hot, making sure to turn off if excessive current or rapid heat occurs. Identify the component that gets hot and investigate it for damage. And then repeat the process after replacing the component.

5.2 Scenario

With your help, Rob was able to find and replace the broken parts. He can program his microcontroller, but the SPI interfaced DAC isn't working as he thinks it should. Briefly describe a technique he could use to debug the circuit.

Answer: A technique to debug the circuit would be to use the oscilloscope to directly observe the SPI signals by probing them. Then by comparing the waveforms and timing to the DAC's datasheet to verify the correct data, chip select, and clock polarity that's being used. This can then indicate whether the problem lies in the firmware or the wiring. If the SPI signals leaving the microcontroller are correct, then the problem is most likely the DAC, the configuring, wiring, or how it's interpreting the received data. If the signals are incorrect, then the issue lies in the microcontroller firmware or SPI configuration.

6 Scenario

You are a test engineer for a drone manufacturer. The flight-ops team describes incidents where drones crash after an aggressive flight maneuver. You think the high motor load during maneuvers is causing the microcontroller to brown out. How could you test this hypothesis?

Answer: First to test it, I would monitor the microcontroller's supply voltage with an oscilloscope while reproducing the aggressive flight maneuver. This would be done by connecting an oscilloscope probe across the

microcontroller's supply and ground. Then configuring the oscilloscope to capture the supply voltage. Then reproduce the aggressive maneuver under the controlled conditions. During the maneuver monitor the supply voltage during the period of maximum motor load, and compare the MCU supply voltage during normal operations versus during the aggressive maneuver. Looking for the voltage dip or transient that approaches or falls below the microcontroller's minimum operating voltage. If the MCU supply voltage drops below its required operating voltage while the drone is performing the crash prone maneuver, then it would support the brown out hypothesis.

7 Practice

Construct this simple, passive low-pass filter on a breadboard.

7.1 Practice

Calculate the theoretical cutoff frequency of this filter. Show your calculations.

Answer:

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2 * \pi(1000\Omega)(100 * 10^{-9}F)} = \frac{1}{(2 * 10^{-4} * \pi)} = 1.592kHz$$

7.2 Practice

Write a Python script to collect data on the frequency response of the filter. Starting below the cutoff frequency and ending above, use SCPI to iterate the function generator through a series of frequencies and acquire the attenuation from the oscilloscope. Capture at least 32 data points. Save the data in a file and use matplotlib to create a Bode plot for the circuit with (correct) labeled axes, a title, and a marker of the experimental cutoff frequency. The Bode plot should be in the report.

Table 1: Experimental Low-Pass Filter Frequency Response

Frequency (Hz)	Vout (VPP)	Gain (dB)
116.01553017399715	0.216	0.20342591706476393
134.59603241553643	0.212	0.041068112621174886
156.15230060004964	0.214	0.12262636102996372
210.17480113324893	0.216	0.20342591706476393
243.8354098268829	0.212	0.041068112621174886
282.88694346259695	0.21	-0.041263211275468364
328.1927872511474	0.212	0.041068112621174886
380.7546021222372	0.21	-0.041263211275468364
441.7344703140069	0.208	-0.1243824066986226
512.4805876960936	0.204	-0.29304575743587824
594.5570708544394	0.2	-0.4650491926742283
689.7785379387658	0.196	-0.6405276788243319
800.2502278161052	0.19	-0.9105770868972733
928.4145445194745	0.184	-1.1892926457631239
1077.1050560367692	0.176	-1.5753957496708573
1249.6091412919868	0.168	-1.979463471436595
1449.7406703726315	0.16	-2.403249452835357
1681.9243248808689	0.158	-2.5125073668654
1951.2934226359623	0.138	-3.688067377929122
2263.8034095214466	0.126	-4.4782382036025945
2626.3635276533328	0.116	-5.196489321415484
3046.989570903508	0.104	-6.144982319978246
3534.981105030106	0.088	-7.595995662950481
4101.1270705513	0.08	-8.42384936611498
4757.944314009409	0.072	-9.338999177328486
5519.954321281573	0.064	-10.362049626276109
6404.004271197283	0.056	-11.521888565829846
7429.63950759495	0.05	-12.506249019233476
8619.535664753032	0.044	-13.616595576230106
10000.0	0.04	-14.444449279394604

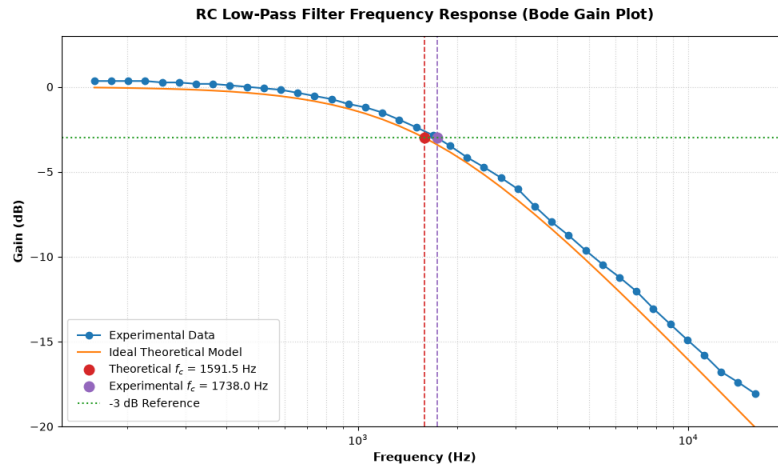


Figure 2: RC Low-Pass Filter Frequency Response Bode Gain Plot (Figure_1.png)

7.3 Practice

Measure and document the resistor and capacitor values using the multimeter and use the values to calculate the actual cutoff frequency. Does this match the frequency from the Bode plot? If not, what might account for discrepancies?

Capacitor = 107.5 nF

Resistor = 1.036k Ω

Cutoff Frequency =

$$f_c = \frac{1}{2\pi RC} = \frac{1}{2 * \pi(1036\Omega)(107.5 * 10^{-9}F)} = 1.429kHz$$

While the actual cut off frequency very closely follows the theoretical expectations, however it does not match the frequency from the Bode plot. This is due to the actual discrepancies in the resistor and capacitor, which were 1.036k Ω and 107.5nF respectively, while the theoretical values were 1k Ω and 100nF respectively. This small differences can be caused from the component tolerances, measurement tool accuracy, or stray resistance and capacitance from the breadboard and wiring. Even though there were some discrepancies, the low pass filter worked as expected.

A Scenario

Listing 1: Python script used to collect the low-pass filter frequency response.

```
# Low-pass filter frequency response
# DG1032 FUNCTION GENERATOR + MS02302A OSCILLOSCOPE
# ONLY CHAN1 IS USED

# Imports
import numpy as np
import pyvisa
import math
import time
import csv
import matplotlib.pyplot as plt

#User settings
GEN_ADDRESS = "TCPIP::169.254.213.175::INSTR"      # DG1032, function generator
SCOPE_ADDRESS = "TCPIP::169.254.43.160::INSTR"    # MS02302A, oscilloscope

# Frequency sweep
START_FREQ = 100 # Starting frequency of the sweep
STOP_FREQ = 10000 # Ending frequency of the sweep
NUM_PTS = 32 # Number of frequency measurements to collect

# Function generator
VOLTAGE_AMPLITUDE = 2.0      # VPP, amplitude of sine wave produced by the
                             # functiongenerator in VPP
DC_OFFSET = 0.0             # V, DC offset of the function generator in volts

# Theoretical circuit values
R = 1000                    # Resistance in ohms
C = 0.1e-6                  # Capacitance in farads

#Theoretical cutoff
theoretical_cutoff = 1 / (2 * math.pi * R * C)

# Print theoretical cutoff
print(
    f"\nTheoretical_cutoff_frequency_="
    f"{theoretical_cutoff:.2f}_Hz"
)

# VISA setup
rm = pyvisa.ResourceManager('@py') # Creates a PyVISA resource manager using the Python
VISA backend

generator = rm.open_resource(GEN_ADDRESS) # Opens communication with the function
generator
scope = rm.open_resource(SCOPE_ADDRESS) # Opens communication with the oscilloscope

generator.timeout = 5000 # Sets function generator communication timeout to 5000 ms
scope.timeout = 5000 # Sets the oscilloscope communication timeout to 5000 ms

# Function generator setup
generator.write("*RST") # Resets function generator to default settings

time.sleep(0.5) # Waits for 0.5 seconds for function generator to reset

# Configure:
generator.write(
    f"APPL:SIN_" # Sine wave
```

```

    f"{START_FREQ}," # 100 Hz initial frequency
    f"{VOLTAGE_AMPLITUDE}," # 2 VPP, sets the output amplitude
    f"{DC_OFFSET}") # 0V DC offset

time.sleep(0.5) # Waits for setting to take effect

# Turn output ON
generator.write("OUTP_ON")

time.sleep(0.5) # Gives generator time to stabilize

# Oscilloscope set uscope# only CH1
scope.write("*RST") # Resets oscilloscope

time.sleep(0.5) # Waits for it to reset

# Enable CH1
scope.write(":CHAN1:DISP_ON") # Turns CH1 on

scope.write(":CHAN1:COUP_DC") # Sets CH1 to DC coupling

# Vertical scale
scope.write(":CHAN1:SCAL_0.05") # Gives better vertical resolution

scope.write(":CHAN1:OFFS_0") # Sets vertical offset to 0 V

# Trigger
scope.write(":TRIG:EDGE:SOUR_CHAN1") # Sets CH1 as trigger source

scope.write(":TRIG:EDGE:SLOP_POS") # Trigger slope to positive

scope.write(":TRIG:EDGE:LEV_0") # Sets trigger level to 0 V

scope.write(":MEAS:CLE") # Clears old measurements

time.sleep(0.5) # Gives time to update

# Creates 32 data points
frequencies = np.logspace(
    np.log10(START_FREQ),
    np.log10(STOP_FREQ),
    NUMPTS
)

# Arrays for measurements
valid_freqs = []

vout_values = []

# Frequency sweep
print("\n===== ")

print("MEASUREMENTS")

print("===== ")

print(
    "Frequency(Hz) | Vout(VPP) "
)

print("----- ")

```

```

for f in frequencies:

    # Generator Frequency
    # Sets the current frequency to function generator
    generator.write(
        f"FREQ_{f}"
    )

    # Give generator a short time to settle
    time.sleep(0.20)

    # Set Oscilloscope time scale
    # Approximately two periods across the screen
    time_per_div = max(
        0.2 / f,
        5e-6
    )

    scope.write(
        f":TIM:SCAL_{time_per_div}" # Sends time scale to o
    )

    # Starts waveform acquisition
    scope.write(":RUN")

    time.sleep(0.25) # Waits for waveform to update

    # -----
    # First measurement
    #
    # Discard the first measurement after changing frequency
    # so that the scope has time to update its waveform.
    # -----

    try:

        scope.query(
            ":MEASure:VPP?_CHANnel1"
        )

    # Handles timeout from
    except pyvisa.VisaIOError:

        print(
            f"{f:10.2f}_Hz_|_"
            f"Measurement_timeout"
        )

        continue

    # Attempts acutal VPP measurement
    try:

        # Requests CH1 peak-to-peak voltage
        vout_string = scope.query(
            ":MEASure:VPP?_CHANnel1"
        ).strip()

        vout = float(vout_string)

```



```

# Handles communication errors
except (pyvisa.VisaIOError, ValueError):

    print(
        f"{f:10.2f}_Hz_|_"
        f"Measurement_failed"
    )

    continue

# Check Measurement# Checks if the measured voltage is valid
if vout <= 0 or vout >= 1e30:

    print(
        f"{f:10.2f}_Hz_|_"
        f"Invalid_Vout=_{vout}"
    )

    continue

# Store valid measurements
valid_freqs.append(f)

vout_values.append(vout)

# Convert to numpy arrays
valid_freqs = np.array(valid_freqs)

vout_values = np.array(vout_values)

# Checks data was collected
if len(valid_freqs) < 3:

    print(
        "\nERROR: Not enough valid measurements."
    )

    generator.write("OUTP_OFF") # Turns generator output off

    generator.close() # Closes generator connection

    scope.close() # Closes oscilloscope connection

    rm.close() # Closes VISA resource manager

    raise SystemExit # Stops program

# Determine low frequency passband reference
# The first point in your previous data was an outlier.
#
# Therefore, don't use the first point to establish the
# passband reference.
#
# Use the next several low-frequency points.

# Determines how many low-frequency measurements to Use
# A maximum of 10 measurements are used
reference_points = min(
    10,
    len(vout_values) - 1

```

```

)

# Calculate median Vout of the low-frequency points
# These points represent the filter's passband
reference_vout = np.median(
    vout_values[
        1:reference_points + 1
    ]
)

#Calculate experiemental gain
gain_db = 20 * np.log10(
    vout_values / reference_vout
)

# Print complete results
print("\n=====")

print("LOW-PASS_FILTER_RESULTS")

print("=====")

print(
    f"Passband_reference="
    f"{reference_vout:.4f}_VPP"
)

print(
    f"Theoretical_cutoff="
    f"{theoretical_cutoff:.2f}_Hz"
)

print("-----")

print(
    "Frequency(Hz)|"
    "Vout(VPP)|"
    "Gain(dB)"
)

print("-----")

for f, v, g in zip(
    valid_freqs,
    vout_values,
    gain_db
):

    # Gain is included on the same result line

    print(
        f"{f:14.2f}|"
        f"{v:10.4f}|"
        f"{g:8.2f}"
    )

# Find experimental -3 dB cutoff
cutoff_gain = -3.0

experimental_cutoff = None # Assume no cutoff found

```

```

# Loop through neighboring gain measurements
for i in range(
    len(gain_db) - 1
):

    g1 = gain_db[i] # Gets current gain measurement

    g2 = gain_db[i + 1] # Gets next gain

    # Check for crossing of -3 dB
    if (
        g1 >= cutoff_gain
        and
        g2 <= cutoff_gain
    ):

        f1 = valid_freqs[i] # Gets first frequency around crossing

        f2 = valid_freqs[i + 1] # Gets second frequency around crossing

        # Log-frequency interpolation, converts to log scale
        log_f1 = np.log10(f1)

        log_f2 = np.log10(f2)

        # Calculates how far between 2 points the -3dB level occurs
        fraction = (
            (cutoff_gain - g1)
            /
            (g2 - g1)
        )

        # Interpolates between the two logarithms
        log_fc = (
            log_f1
            +
            fraction *
            (log_f2 - log_f1)
        )

        # Converts interpolated logarithm back into Hz
        experimental_cutoff = (
            10 ** log_fc
        )

        break

print("\n=====")

print("CUTOFF FREQUENCIES")

print("=====")

print(
    f"Theoretical cutoff frequency = "
    f"{theoretical_cutoff:.2f} Hz"
)

if experimental_cutoff is not None:

```

```

print(
    f"Experimental_cutoff_frequency="
    f"{experimental_cutoff:.2f}Hz"
)

# Calculate percent difference
percent_difference = (
    abs(
        experimental_cutoff
        -
        theoretical_cutoff
    )
    /
    theoretical_cutoff
    * 100
)

print(
    f"Percent_difference="
    f"{percent_difference:.2f}%"
)

else:

    print(
        "Experimental_cutoff_frequency"
        "could_not_be_determined."
    )

print("=====")

# Save CSV
# Opens/Creates CSV file
with open(
    "lowpass_data.csv",
    mode="w",
    newline=""
) as file:

    writer = csv.writer(file) # Creates a CSV filter

    # Write column headings
    writer.writerow(
        [
            "Frequency(Hz)",
            "Vout(VPP)",
            "Gain(dB)"
        ]
    )

    # Loop through measured data
    for f, v, g in zip(
        valid_freqs,
        vout_values,
        gain_db
    ):

        # Write one measurement row to CSV

```

```

        writer.writerow(
            [
                f,
                v,
                g
            ]
        )

# Create Ideal Theoretical Frequency response
theoretical_freqs = np.logspace(
    np.log10(START_FREQ),
    np.log10(STOP_FREQ),
    500
)

# -----
# Ideal RC low-pass magnitude
#
#  $|H(f)| =$ 
#
#  $\frac{1}{\sqrt{1 + (f/f_c)^2}}$ 
# ----- #

theoretical_magnitude = (
    1 /
    np.sqrt(
        1 +
        (
            theoretical_freqs
            /
            theoretical_cutoff
        ) ** 2
    )
)

# Convert ideal magnitude to dB
theoretical_gain_db = (
    20 *
    np.log10(
        theoretical_magnitude
    )
)

# Bode plot
plt.figure(
    figsize=(9, 6)
)

# Experimental response
plt.semilogx(
    valid_freqs,
    gain_db,
    marker='o',
    linestyle='--',
    label="Experimental"
)

```

```

# Ideal theoretical response
plt.semilogx(
    theoretical_freqs,
    theoretical_gain_db,
    linestyle='--',
    label="Ideal Theoretical"
)

# -3 dB reference
plt.axhline(
    y=-3,
    linestyle=':',
    label="-3 dB"
)

# Theoretical cutoff
plt.axvline(
    x=theoretical_cutoff,
    linestyle='--',
    label=(
        f"Theoretical cutoff = "
        f"{theoretical_cutoff:.1f} Hz"
    )
)

# Experimental cutoff
if experimental_cutoff is not None:
    plt.axvline(
        x=experimental_cutoff,
        linestyle=':',
        label=(
            f"Experimental cutoff = "
            f"{experimental_cutoff:.1f} Hz"
        )
    )

# Mark experimental -3 dB point
plt.plot(
    experimental_cutoff,
    -3,
    marker='o',
    markersize=8
)

# Plot labels
plt.xlabel(
    "Frequency (Hz)"
)

plt.ylabel(
    "Gain (dB)"
)

plt.title(
    "Low-Pass Filter Frequency Response"
)

# Grid
plt.grid(

```

```
        True,
        which="both"
    )

# Legend
plt.legend()

# Layout
plt.tight_layout()

# Display
plt.show()

# Turn off and close instruments
generator.write("OUTP_OFF")

generator.close()

scope.close()

rm.close()
```